
Ответы на вопросы Задания 03

1. Пояснение понятия «событие программного объекта»

Событие программного объекта — это специальный сигнал или уведомление, которое объект генерирует, когда происходит определённое действие или изменение состояния.

Примеры:

- Пользователь нажал кнопку → кнопка генерирует событие `click`.
- В базе данных появилась новая запись → объект базы данных может сгенерировать событие `insert`.

Основная идея: объект сообщает другим частям программы, что что-то произошло, и эти части могут на это «реагировать», не контролируя постоянно состояние объекта.

2. Механизм генерации и обработки событий в C#

В C# события основаны на **делегатах**. Делегат — это тип, который описывает метод с определённой сигнатурой.

Пример работы:

1. **Объявление делегата** — описывает тип функции-обработчика события.

```
public delegate void Notify(string message);
```

2. **Объявление события:**

```
public event Notify OnChange;
```

3. **Генерация события (вызов обработчиков):**

```
OnChange?.Invoke("Data updated");
```

4. **Подписка на событие (обработчик):**

```
myObject.OnChange += (msg) => Console.WriteLine(msg);
```

Принцип работы: объект вызывает `Invoke` события — все подписанные обработчики выполняются автоматически.

3. Реализация механизма генерации и обработки событий на JS или C++

JavaScript

В JS события можно реализовать через паттерн **Observer / Event Emitter**:

```
class DB {
  constructor() {
    this.data = [];
    this.events = {}; // хранение обработчиков
  }

  on(event, handler) {
    if (!this.events[event]) this.events[event] = [];
    this.events[event].push(handler);
  }

  emit(event, payload) {
    if (this.events[event]) {
      this.events[event].forEach(handler => handler(payload));
    }
  }

  insert(row) {
    this.data.push(row);
    this.emit('insert', row); // генерация события
  }
}

const db = new DB();
db.on('insert', row => console.log('Added:', row));
db.insert({id:1, name:"Alice", bday:"2000-01-01"});
```

C++

В C++ часто используют паттерн наблюдателя (Observer):

```
#include <iostream>
#include <vector>
#include <functional>
using namespace std;
```

```

class DB {
    vector<function<void(int)>> listeners;
public:
    void onInsert(function<void(int)> handler) { listeners.push_back(handler); }

    void insert(int id) {
        // вставка в "таблицу"
        for(auto &handler: listeners) handler(id); // уведомление всех
        подписчиков
    }
};

int main() {
    DB db;
    db.onInsert([](int id){ cout << "Inserted: " << id << endl; });
    db.insert(1);
}

```

4. Встроенный механизм событий в Node.js

В Node.js для генерации и обработки событий используется встроенный модуль `events` с классом `EventEmitter`.

Принцип работы:

1. Класс `EventEmitter` позволяет объекту **генерировать события** (`emit`).
2. Другие части программы могут **подписываться на события** (`on`).
3. Когда событие вызывается (`emit`), Node.js последовательно выполняет все подписанные обработчики.

Пример:

```

const EventEmitter = require('events');
const emitter = new EventEmitter();

// Подписка на событие
emitter.on('message', (msg) => {
    console.log('Received:', msg);
});

// Генерация события
emitter.emit('message', 'Hello World');

```

Таким образом, Node.js реализует **асинхронную модель событий**, где события могут происходить в любом порядке, а обработчики вызываются автоматически при их возникновении.
